

**PROJECT SYNOPSIS ## VGU STUDENT MANAGEMENT
SYSTEM: A ROLE-BASED WEB PORTAL FOR ACADEMIC
ADMINISTRATION**

1. COVER PAGE

Project Title: VGU Student Management System: A Role-Based Web Portal for Academic Administration

Course: Bachelor of Technology (B.Tech) in Computer Science & Engineering **Academic Year:** 2026

Submitted By: MCA / B.Tech CSE Final Year Students **Supervisor / Guide Name:** [Guide Name Placeholder]

Department: Department of Computer Science & Engineering **Institution:** Vivekanand Global University (VGU)

2. CERTIFICATE

This is to certify that the project entitled "**VGU Student Management System**" is a bonafide work carried out by the student(s) in partial fulfillment of the requirements for the award of the degree of Bachelor of Technology in Computer Science & Engineering during the academic year 2025–2026.

Signature of Guide: _____ **Name & Designation:** _____ **Date:**

Signature of Head of Department: _____ **Name & Designation:**
_____ **Date:** _____

3. ACKNOWLEDGEMENT

We express our deep sense of gratitude to the Department of Computer Science & Engineering, Vivekanand Global University, for providing the necessary facilities and environment to carry out this project. We are highly indebted to our project guide for their invaluable guidance, constant supervision, and constructive feedback throughout the project lifecycle.

Finally, we thank our parents, peers, and laboratory staff members who directly or indirectly assisted us in finishing this research work.

4. TABLE OF CONTENTS 1. Cover Page 2. Certificate 3. Acknowledgement 4. Project Title & Abstract 5. Introduction & Problem Statement 6. Objectives & Scope 7. Analysis of the Existing vs. Proposed System 8. System Feasibility Study 9. System Requirements Specification 10. Functional & Non-Functional Requirements 11. System Architecture & Modules 12. Advantages & Limitations 13. Future Scope & Conclusion 14. References

5. PROJECT TITLE VGU Student Management System: A Role-Based Web Portal for Academic Administration

6. ABSTRACT

The VGU Student Management System is a role-aware web-based administration portal designed to streamline academic workflows, manage user roles, track attendance, coordinate examinations, and handle financial fee schedules. Built using Python 3.12 and Django 5.x, the system replaces manual paper-based processes and isolated spreadsheets with a unified relational database powered by SQLite.

The application implements three primary user roles: Administrator (full system oversight), Teacher (subject allotment, marks upload, and attendance management), and Student (viewing academic performance, personal attendance, schedules, and fee payments). Security is enforced through custom Django middleware, decorators (@admin_required, @teacher_or_admin_required), and Django REST Framework's token-based authentication (SimpleJWT). Performance optimizations, such as database query caching and N+1 query elimination via Django's `select_related()` and `prefetch_related()`, ensure fast response times. The interface features a responsive light/dark design theme powered by Bootstrap 5.3, dynamic Chart.js dashboards, and a custom Javascript toast notification framework.

7. INTRODUCTION

Educational institutions require robust information systems to manage student enrollments, faculty workloads, daily attendance registers, examinations, and fee collections. Traditional administration systems are often fragmented, relying on separate sheets or legacy software that lacks security and performance optimization.

The VGU Student Management System resolves these challenges by integrating all academic processes into a secure, responsive, and performance-optimized Django portal. It provides real-time dashboards for administrators, teachers, and students, allowing transparent tracking of academic records, timetables, and assignments.

8. PROBLEM STATEMENT

Existing academic workflows suffer from:

- Inefficient Manual Recording:** Paper records and generic spreadsheets are prone to data duplication, loss, and synchronization delays.
- Lack of Role Segregation:** Faculty, students, and admins lack secure, role-restricted portals to manage data safely.
- Database Performance Bottlenecks:** Legacy systems suffer from N+1 query problems where fetching related models (e.g., fetching a student's department or course) results in duplicate database queries, causing slow page loads.
- Poor Interface Responsiveness:** Administrators and users lack mobile-responsive designs and dark-mode alternatives, reducing usability.

9. OBJECTIVES

The system aims to: - Establish a secure, centralized database storing records for students, teachers, departments, and courses. - Implement strict Role-Based Access Control (RBAC) ensuring students cannot alter marks or attendance. - Optimize database execution speed by writing clean Django ORM queriesets using `select_related`. - Modernize the user experience using a responsive theme switcher (Light/Dark), GPU-friendly micro-animations, and animated KPI counters. - Provide a robust REST API with JWT security and Swagger documentation for potential third-party integrations.

10. SCOPE

The VGU Student Management System applies to Vivekanand Global University's computer labs, registrar offices, and student devices. The scope covers: - Student enrollment, profile tracking, and CSV reports generation. - Teacher assignments to subjects and classes. - Daily attendance marking by subject teachers. - Examination scheduling and marks entry. - Semester-wise fee payment recording and receipts. - Live system notifications (Notice Board) and assignments upload/submission portals.

11. EXISTING SYSTEM

The existing system is manual, relying on physical attendance registers and offline Excel sheets. * **Drawbacks:** - High probability of human error during transcript entry. - Zero authorization audits; teachers can access other teachers' marks. - No automated calculation of average marks or attendance ratios. - Hard to verify fee payment compliance.

12. PROPOSED SYSTEM

The proposed system is a centralized web portal built on Python, Django, and SQLite. * **Key Features:** - Automated average calculation for marks and attendance percentages. - Granular permission decorators preventing students and teachers from crossing boundaries. - Live charts showing attendance patterns and department distribution. - RESTful APIs allowing cross-platform mobile access.

13. TECHNOLOGIES USED

- **Programming Language:** Python 3.12
 - **Backend Framework:** Django 5.0.x, Django REST Framework (DRF) 3.14+
 - **Database Engine:** SQLite 3 (relational)
 - **Frontend CSS Framework:** Bootstrap 5.3.2 (responsive styles)
 - **Frontend Interaction:** jQuery 3.7.1, Bootstrap Icons 1.11.3
 - **Data Visualization:** Chart.js (CDN)
 - **Token Security:** simplejwt (JSON Web Tokens)
 - **Documentation:** drf-yasg (Swagger/OpenAPI interface)
-

14. SOFTWARE REQUIREMENTS

- **Operating System:** Windows 10/11, macOS, or Linux (Ubuntu 22.04 LTS recommended)
 - **Python Version:** Python 3.12.x
 - **Development Tool:** PyCharm, VS Code, or Antigravity IDE
 - **Database Console:** DB Browser for SQLite / SQLite3 CLI
 - **Web Browser:** Google Chrome 120+, Mozilla Firefox 115+, or Microsoft Edge
-

15. HARDWARE REQUIREMENTS

- **Processor:** Intel Core i3 (4th Gen) / AMD Ryzen 3 or higher
 - **RAM:** Minimum 4 GB (8 GB recommended for development environment)
 - **Hard Disk Space:** 10 GB free space (SSD preferred)
 - **Display Resolution:** 1366x768 minimum; supports full 1080p responsive layouts
-

16. FEASIBILITY STUDY

1. **Technical Feasibility:** Python and Django provide robust, secure, and ready-made modules for authorization and data handling. SQLite is highly compatible and lightweight, requiring no external server processes. Thus, the project is technically feasible.
 2. **Economic Feasibility:** The project uses open-source technologies (Python, Django, Bootstrap, SQLite). No software license fees are required, making the project highly cost-effective and economically viable.
 3. **Operational Feasibility:** Administrators and faculty require minimal training, as the interface is self-explanatory and responsive. Students can track their performance on their own phones, showing high operational feasibility.
-

17. FUNCTIONAL REQUIREMENTS

- **Authentication Module:** Secure login, password hashing (PBKDF2), role detection, password change forms, and logout routes.
 - **Student Profile Management:** Admin CRUD interface for students. Automatically creates a linked User credentials account.
 - **Teacher Assignment Module:** Teacher registry containing qualifications and contact data, linked to specific academic subjects.
 - **Attendance Register:** Subject-specific daily attendance records marked by assigned teachers.
 - **Exams and Grading System:** Exam creation, max marks definition, and marks entry. Auto-calculates percentages and grades.
 - **Billing System:** Recording fee categories and student payments, tracking balances, and generating receipts.
-

18. NON-FUNCTIONAL REQUIREMENTS

- **Security:** Hashing user passwords, securing CSRF tokens, preventing SQL injections via Django ORM, and filtering REST API views with JWT.
- **Performance:** Page response time under 1.5 seconds. N+1 queries eliminated using `.select_related()` for related fields.
- **Usability:** Light/dark modes, custom toast alert panels, and sidebars optimized for mobile, tablet, and desktop viewports.
- **Scalability:** Easily migrates to PostgreSQL or MySQL database engines for high concurrent access.

19. SYSTEM MODULES

1. **Auth & Profile Module:** Manages logins, roles (admin, teacher, student), profile photos, and token authorization.
 2. **Academics Module:** Governs Departments, Courses (B.Tech, M.Tech, etc.), and Subjects.
 3. **Daily Attendance Module:** Faculty interface to mark presence/absence and students interface to check their attendance percentage.
 4. **Grading Module:** Faculty marks upload panel and auto-generation of grades (A+, A, B+, B, C, D, F).
 5. **Notice Board Module:** Board for administrators to post news.
 6. **Billing & Fees Module:** Tracking payments, dues, and generating receipt codes.
 7. **Assignments Module:** Faculty files distribution and students submission portal.
 8. **Reports Module:** Graphical reports displaying attendance distributions and subject-wise averages.
-

20. ADVANTAGES

- **Zero N+1 Query Overhead:** Efficient querysets ensure quick page loads.
 - **Flexible UI Theme:** Smooth light/dark theme switcher with saved user preferences.
 - **Comprehensive API:** Complete Swagger documentation for modern app connectivity.
 - **Toast Notifications:** Interactive messages rather than disruptive native alerts.
-

21. LIMITATIONS

- **File Storage System:** Currently saves uploaded assignments on the local server disk instead of cloud buckets (S3).
 - **Payment Integration:** Lacks direct payment gateway integration (Stripe/PayPal), requiring admins to manually record receipts.
 - **Real-time Chat:** Communication is asynchronous (Notice Board) rather than instant messaging.
-

22. FUTURE SCOPE

- Integrate Razorpay / Stripe gateway for online automated student fee collections.
 - Implement Django Channels to support real-time instant chat and class forum features.
 - Move file storage to Amazon S3 or Google Cloud Storage.
 - Develop a companion mobile application using Flutter connecting to the DRF endpoints.
-

23. CONCLUSION

The VGU Student Management System successfully implements a secure, performance-optimized portal for university administration. By combining Python's Django framework with responsive Bootstrap elements, the system eliminates paper registers, prevents unauthorized access, and accelerates the page loading speeds through database queryset optimization. It provides a solid model for B.Tech CSE students in database design, web application development, and software engineering.

24. REFERENCES

1. J. McGaw, *Beginning Django: Web Application Development and Deployment with Python*, Apress, 2020.
2. M. Richardson and S. Ruby, *RESTful Web Services*, O'Reilly Media, Inc., 2007.
3. Bootstrap Contributors, *Bootstrap 5.3 Documentation: Theme Colors & Customization*, 2023. Available: <https://getbootstrap.com>
4. Django Software Foundation, *Django 5.0 Release Notes & Performance Optimization*, 2024. Available: <https://docs.djangoproject.com>